# Small Models, Big .NET Energy

AI , Agents, Foundry Local and more . . .



**Bruno Capuano**

Principal Cloud Advocate, Microsoft

One focused model can make a small task feel like a complete product feature



# Local speech to text from a file

SAMPLE RECORDING

“So, um, I need to send the the report by Friday. No, wait, make that Thursday at three thirty p.m. Please update the project notes before the meeting.”

A short English audio file goes to a local speech model in Foundry Local

TEXT AS IT ARRIVES



Words show up while the audio is read

FINAL RESULT

**Read the full transcript once the audio is done**

The live text is a preview, the final transcript is the result

# Live text first, final transcript last

```
// Stream: print text as it arrives
await foreach (var item in stream)
    if (item is SpeechSegmentItem segment)
        Console.Write(segment.Text);

// Final: read the transcript after Stop
using var finalResponse = await stream.FinalResponse;
if (finalResponse.FinishReason ≠ FinishReason.Stop)
    throw new InvalidDataException( ... );

var transcript = finalResponse
    .OfType<SpeechResultItem>().SingleOrDefault()?.Text;
```

# Demo 1

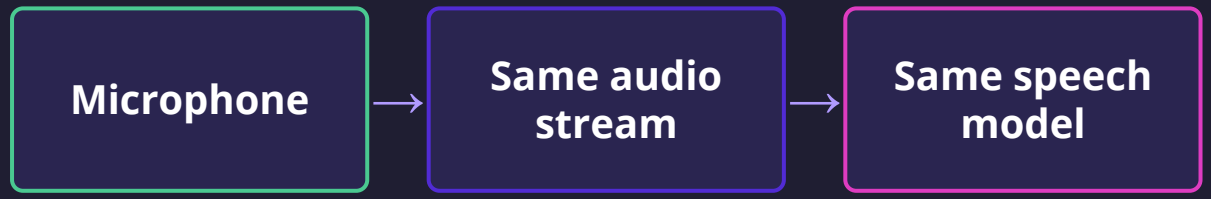
Local speech to text from an audio file



# Same code, live microphone

WHAT CHANGES

The microphone replaces the audio file. The model and the code stay the same




Microphone audio flows into the same audio stream

LIVE OUTPUT

Text appears while you speak

STOP AND FINAL RESULT

**Press ENTER to stop, then read the final transcript**

No microphone shows a clear error, and silence prints Nothing was heard

# Demo 2

Live local speech to text from the microphone



# Raw speech is not clean text

FILLERS

## “um” · “the the”

Um, so and repeated words like the the stay in the transcript

SELF-CORRECTIONS

## “No, wait, make that Thursday”

No, wait, make that Thursday leaves both versions in the text

SPOKEN FORMATS

## “three thirty p.m.”

Three thirty p.m. arrives as words, with little or no punctuation

# Raw text, cleaner wording

“So, **um**, I need to send **the the** report by Friday. No, wait, make that Thursday at **three thirty p.m.** Please update the project notes before the meeting.”

→

“So, I need to send the report by Friday. No, wait, make that Thursday at **3:30 p.m.** Please update the project notes before the meeting.”

## RAW TRANSCRIPT

**Printed exactly as recognized**

ORIGINAL

## S1-MINI CLEANUP

**Fillers, punctuation and number formatting**

CLEANED

**Cleanup tidies the wording. It does not fix misheard words or translate**

# Clean up with clear settings

STYLING
**Semi-formal**

STRUCTURE
**Prose**

CONTEXT
**General**

OUTPUT
**Printed separately**

```
// Step 1: same speech step as Demo 1, then unload

// Step 2: explicit controls
//   Styling:   semi-formal
//   Structure: prose
//   Context:   general

// Step 3: normalize the actual recognized text

// Step 4: print raw and cleaned separately

// Walk through Program.cs in the editor
```

# Demo 3

Clean up a transcript with a small local model



# Foundry Local runs both models

English WAV → Nemotron speech → Raw transcript → S1-Mini cleanup → Cleaned transcript



## Foundry Local manages both models

Speech unloads first, then S1-Mini loads. Both are set up before recording

# Demo 4

The same cleanup, managed by Foundry Local



# Decision models: one answer, not chat

**WHAT IT DOES**

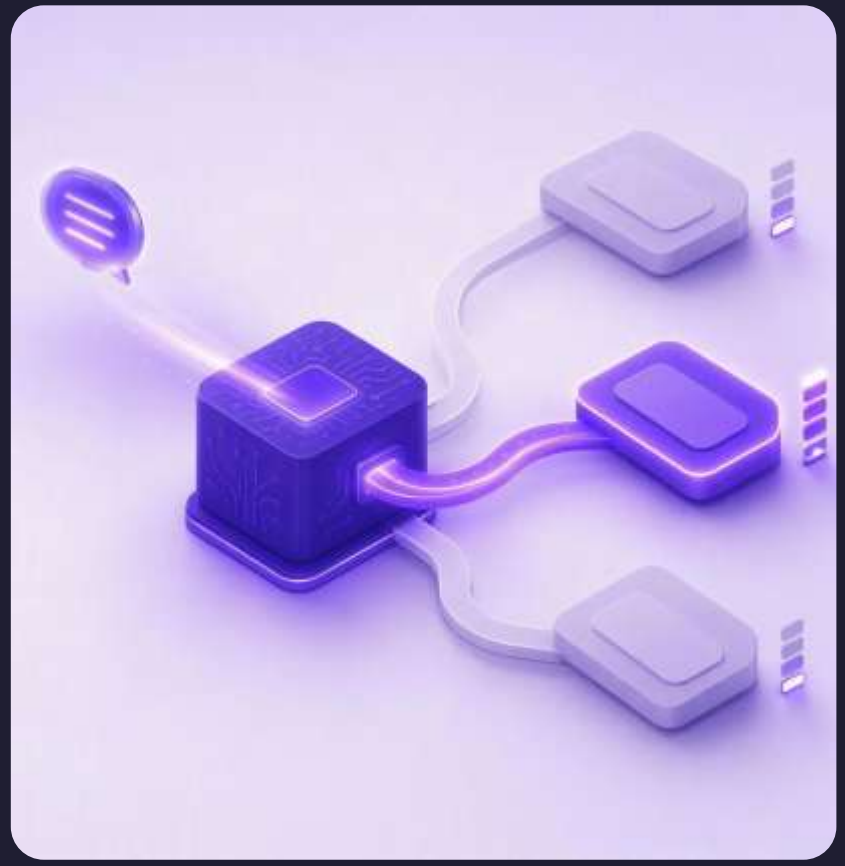
Chooses one answer from the options your app supplies

**WHAT YOU GET**

The choice, a confidence score and a probability for each option

**WHERE TO USE THEM**

In Ollama today. Coming soon to Foundry Local and MEAI (Microsoft.Extensions.AI)



# Text and options in, one choice out

```
const string input = "Please correct my invoice because it was charged twice.";
const string instructions = "Which team should handle this request?";
var options = new Dictionary<string, string?> {
    ["billing"] = "Invoices, charges, payments, or account balances",
    ["support"] = "Technical product problems and troubleshooting" };
var result = await client.ChooseAsync(input, instructions, options, shutdown.Token);
// Sample output from one run (scores vary)
// result.Choice        → "billing"
// result.Confidence    → 99.2%
// result.Probabilities → billing 99.6%, support 0.4%
```

# Demo 5

Local decisions with Ollama



# Different layers, different jobs

### RUNTIMES

Foundry Local and Ollama run the live local models. llama.cpp is another runtime, not used by these demos.

### TASK-SPECIFIC MODELS

Nemotron for speech, S1-Mini for cleanup, and nimble:latest for bounded decisions

### APPLICATION FRAMEWORKS

Microsoft.Extensions.AI supplies abstractions. Agent Framework orchestrates agents. This direct pipeline needs neither an agent nor a third runtime.

# Put it all together: a local .NET game

## GAMEPLAY: LOCAL DECISIONS WITH OLLAMA

**.NET game**
Sends world state and five legal movement choices

→

**Ollama Decisions**
nimble:latest returns one action with probabilities

→

**.NET game**
Validates the action, then applies a bounded movement burst

## MISSION CONTROL: VOICE WITH FOUNDRY LOCAL

**Microphone**
Audio, only when started on purpose

→

**Foundry Local**
Speech model turns audio into a transcript

→

**.NET parser**
Exact START, PAUSE or STOP

→

**Mission state**
Start, pause or end the run

**.NET owns autofire, collisions, waves and rendering. Voice never goes through Ollama**

# Put it all together

ACTIVE PLAY

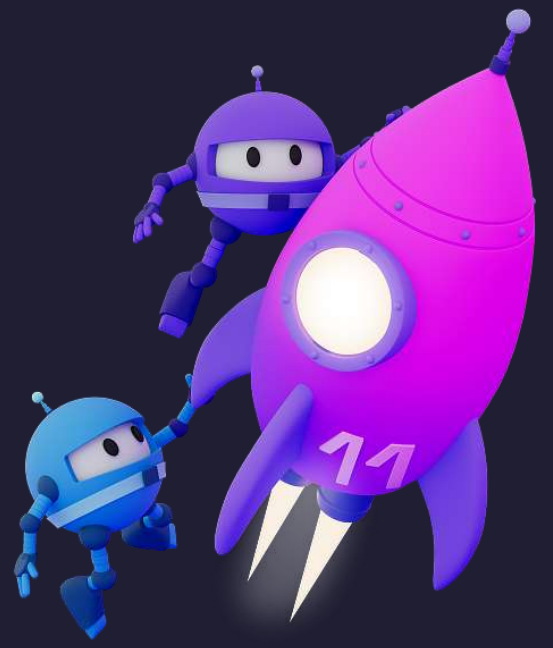



PAUSED: ENTER TO RESUME




**A .NET game with local decisions and voice controls**

Hear the signal
Clean the intent
Decide within bounds
Play it all together
**All LOCAL + .NET**



Thank you

Questions?

